

AI VISION VIAD VISUALLY IMPAIRED ASSISTIVE DEVICE

Students:

Rayan Altawijari	431107823
Faris Alsuhibani	431107834
Mohammed Alsharekh	431108100
Mohaya Almutairi	431107802
Abdullah Alfayez	431108148

Supervisor:

Dr. Ali Mustafa Qamar Khan

A project report submitted to Qassim University in partial fulfillment of the requirements for the degree of B.Sc. in Computer Science

Department of Computer Science
Qassim University
1447/1448 (2025/2026)

CERTIFICATE

It is certified that this project report has been prepared and written under my direct supervision and guidance. This project report is approved for submission for its evaluation.

Ali Mustafa Qamar Khan

DEDICATION

To all those who have contributed to the production of this report, this document is respectfully dedicated, success of this project. Their loyalty, dedication and efforts remain remarkable in the premises of many focused organizations, commitment played a key role towards the achievement of the goals and objectives.

Rayan Altawijari

Faris Alsuhibani

Mohammed Alsharekh

Mohaya Almutairi

Abdullah Alfayez

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to Allah, for granting us the health, patience, and ability to complete this project.

We would like to express our deepest gratitude to our supervisor, Dr. Ali Mustafa Qamar Khan, whose guidance, encouragement, and insightful feedback were invaluable throughout every stage of this work.

We are also sincerely grateful to Fikrah Lab at the Applied College, and in particular Dr. Yasser Abdulaziz, for providing the 3D-printing and fabrication facilities used to build the wearable frame, as well as the electronic components and hardware that made this project possible.

Finally, we extend our heartfelt thanks to our families for their constant support and patience, and to the Department of Computer Science at Qassim University for the knowledge and resources that prepared us to carry out this work.



Names Rayan Altawijari

Faris Alsuhibani

Mohammed Alsharekh

Mohaya Almutairi

Abdullah Alfayez

CONTENTS

List of Tables	vii
List of Figures	viii
List of Abbreviations	ix
Abstract	x
Chapter One	1
1. Introduction	1
1.1 Introduction	1
1.2 Project Scope	1
1.3 Aim and Objectives	2
1.4 Motivation	3
1.5 Project Plan and Schedule	4
1.6 Outline of the Report	5
Chapter Two	6
2. Literature Review	6
2.1 Introduction	6
2.2 Background	7
2.2.1 <i>Raspberry Pi Based System</i>	7
2.2.2 <i>Object Detection Models</i>	9
2.2.3 <i>Distance Sensors and Sound Navigation</i>	10
2.2.4 <i>Hybrid Voice Interface</i>	10
2.3 Existing Systems	11
2.3.1 <i>Ultrasonic Cane Systems</i>	12
2.3.2 <i>Camera Based Smart Glasses</i>	12
2.3.3 <i>GPS and Mobile App Based Navigation Systems</i>	13
2.4 Contribution	14
Chapter Three	16
3. Problem Analysis	16
3.1 Introduction	16
3.2 Problem Specification	16
3.3 System Analysis	17
3.3.1 <i>Requirement Collection</i>	17
3.3.2 <i>Requirement Analysis</i>	18
3.4 Implementation and Evaluation Plan	21
3.4.1 <i>Technical Tools and Programming Languages</i>	21
3.4.2 <i>Frameworks</i>	22
3.4.3 <i>Tools</i>	23

3.4.4 Evaluation Methodology	23
Chapter Four	26
4. System Design	26
4.1 Introduction	26
4.2 System Design Specification	26
4.3 Design Architecture	28
Chapter Five	32
5. System Implementation & Integration	32
5.1 Introduction	32
5.2 Implementation and Evaluation Plan	32
5.2.1 Hardware Integration and Wearable Assembly	33
5.2.2 Software Environment and Model Optimization	34
5.2.3 Implementation Objectives	34
5.2.4 Audio Output and Neural Voice Caching	35
5.3 Model Implementation	36
5.3.1 Dataset and Training	36
5.3.2 Training Results	37
5.3.3 Model Selection for Deployment	38
5.4 Software Development and Algorithms	39
5.4.1 The Multithreaded Execution Model	39
5.4.2 Local Detection and the SSD MobileNet Pipeline	40
5.4.3 Semantic Intelligence and API Integration	41
5.5 System Integration and Orchestration	41
5.6 Code Structure and Repository Organization	42
5.7 Challenges and Problem Solving	43
Chapter Six	45
6. System Testing and Result Discussion	45
6.1 Introduction	45
6.2 Testing Description and Test Cases	46
6.2.1 Component Level Validation (Unit Testing)	46
6.2.2 Integration and Stress Testing	47
6.2.3 Real World Navigation Scenarios (Test Cases)	47
6.3 Results and Discussions	49
Chapter Seven	51
7. Conclusion and Future Work	51
7.1 Conclusion	51
7.2 Future Work	52
7.3 FYP Closing Remarks	53

References	54
Appendix	56
Appendix 1	56
Appendix 2	56

LIST OF TABLES

Table 0.1: List of abbreviations	ix
Table 2.1: Features and Limitations of different existing system.....	14
Table 3.1: Functional vs. Non-Functional Requirements of the System	Error!
Bookmark not defined.0	
Table A.1: Hardware Components and Wiring	55

LIST OF FIGURES

Figure 1.1: Gantt Chart for the project	5
Figure 2.1: modern smart glasses designed for visually impaired users	6
Figure 2.2: Raspberry Pi 4 Model B, central processor for the proposed system ..	8
Figure 2.3: Output of an object detection model identifying multiple objects	9
Figure 2.4: Ultrasonic Distance Sensor	10
Figure 2.5: wewalk smart cane	12
Figure 2.6: OrCam MyEye.....	13
Figure 3.1: Use Case Diagram for the AI Smart Glasses System	20
Figure 4.1: System flowchart for the AI Smart Glasses System	29
Figure 4.2: System architecture for the AI Smart Glasses System	30
Figure 5.1: Hardware wiring and component interconnection schema	33
Figure 5.2: training result of yolo model.....	38
Figure 5.3: comparison between different models running on raspberry pi 4[7] ..	38
Figure 6.1: final hardware and results	49

LIST OF ABBREVIATIONS

Table 0.1: List of abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
CSI	Camera Serial Interface
FPS	Frames Per Second
GIS	Geographic Information System
GPIO	General Purpose Input/Output
GPS	Global Positioning System
GPU	Graphics Processing Unit
HC-SR04	Ultrasonic Distance Sensor Module
I/O	Input/Output
LiteRT	Lite Runtime (formerly TensorFlow Lite)
mAP	mean Average Precision
ML	Machine Learning
NLP	Natural Language Processing
OS	Operating System
SBC	Single-Board Computer
SLM	Small Language Model
SSD	Single Shot MultiBox Detector
STT	Speech-to-Text
TFLite	TensorFlow Lite
TTS	Text-to-Speech
UAT	User Acceptance Testing
USB	Universal Serial Bus
VIAD	Visually Impaired Assistive Device
WER	Word Error Rate
WHO	World Health Organization
YOLO	You Only Look Once

ABSTRACT

This project presents the design, implementation, and evaluation of AI Vision VIAD, an AI powered smart glasses system that helps visually impaired users perceive and navigate their surroundings through real time audio feedback. The wearable integrates computer vision, ultrasonic distance sensing, and a hybrid voice assistant on a Raspberry Pi 4B housed in a 3D-printed frame. A head mounted camera captures the scene continuously, a lightweight object detection model identifies common objects and obstacles, and an HC-SR04 ultrasonic sensor measures their distance the two streams are fused so the system can warn the user with both an object label and its proximity for example, "Careful, chair at 50 centimeters."

The system is built around a dual loop, multithreaded architecture. A continuous offline Safety Loop runs an SSD-MobileNet detector through the TensorFlow Lite interpreter and fuses it with the ultrasonic readings to deliver low latency hazard alerts without any internet connection, while a button triggered online Cognitive Loop sends a high resolution frame to the Gemini 2.5 Flash model to generate richer natural language scene descriptions on demand. The device operates fully bilingually in English and Arabic, with the language chosen by voice at startup and applied to all alerts, speech recognition, and assistant responses. Spoken output is produced by a two tier text-to-speech system that plays cached neural voices when available and falls back to the offline espeak-ng engine, ensuring the safety critical audio path is never silent.

A custom YOLOv11-Nano detector was trained as an accuracy baseline (mAP@0.5 of 0.71), but SSD-MobileNet was selected for deployment because it sustained real time performance on the CPU only Raspberry Pi. In testing, the Safety Loop delivered obstacle alerts in roughly 150 milliseconds and the Cognitive Loop returned scene descriptions in about 2.5 seconds, and the sensor fusion design preserved reliable proximity warnings even when the camera alone failed, such as with glass doors or poor lighting. The results demonstrate that an affordable, wearable device combining edge computing with cloud based multimodal AI can provide both immediate safety and on demand environmental understanding, giving visually impaired users greater independence than single sensor assistive tools.

CHAPTER ONE

1. INTRODUCTION

1.1 Introduction

In recent years, artificial intelligence and computer vision have quickly progressed, creating new opportunities for assistive and interactive technologies. This project aims to develop a pair of AI powered smart glasses that can detect and recognize objects in real time, along with a voice chatbot assistant. The system is built to improve user interaction with their environment by providing instant visual information through audio feedback. The main goal is to create an assistive device that boosts user awareness and accessibility by giving spoken feedback about nearby objects and enabling natural voice interaction. This combination of vision and natural language processing can help with both everyday convenience and accessibility, especially for visually impaired users. In the first phase of this project, we will concentrate on planning and system design, defining the necessary hardware and software components, outlining data flow, and setting the stage for later implementation and testing.

1.2 Project Scope

This project covers the design, planning, and setup for a pair of glasses powered by artificial intelligence. The glasses use a camera to spot and name objects around the user. They give feedback right away through voice. The main idea is to build something that helps people stay more aware of their surroundings and connect better with them.

Right now, we are working on a solid concept for how it all fits together. That includes mapping out the system that links the camera part with a chatbot for talking back. We will pick hardware that works well, stays light and easy to carry. Then comes building a basic prototype. It will show how it detects objects using trained models for computer vision. basic voice chatbot will control it.

We plan to test everything Testing will ensures the accuracy of converting the visual information into voice feedback. The performance of the prototype will identify potential improvements.

Overall, the project spotlights the key features of these AI glasses for helping users. It sets up a strong base for later upgrades. Those could include better accuracy in spotting things, easier voice chats, and hardware that fits just right.

1.3 Aim and Objectives

The aim of this project is to design and develop a pair of smart glasses that combine artificial intelligence, computer vision, and a voice based chatbot to assist users in recognizing and interacting with their surroundings. The system is intended to detect objects in real time and provide helpful verbal feedback through an interactive Artificial Intelligence (AI) assistant.

To achieve this aim, the project will:

- Design the system architecture that integrates computer vision and natural language processing modules.
- Develop an object detection model capable of identifying common items in real time.
- Implement a voice chatbot that allows users to communicate naturally with the system.

- Establish a smooth communication link between the vision system and the chatbot for coordinated responses.
- Test and evaluate the prototype to ensure accuracy, responsiveness, and user friendliness.

1.4 Motivation

Sight is one of the most important and valuable blessings we often take for granted, we don't truly appreciate it until we lose it. Around the world, an estimated 295 million people have moderate to severe visual impairment, and about 43 million of them are legally blind [9].

These individuals face difficulties performing tasks that most of us do effortlessly, and some activities are nearly impossible for them. Over time, many inventions have been created to make life easier for the visually impaired, from regular and smart canes that help avoid obstacles to smart glasses. One of the latest breakthroughs in this field is the RayBan Meta glasses, which can describe objects held in front of them. While these technologies do help with everyday tasks, they are not without problems.

First, the cost of such devices can be very high; not everyone can afford smart glasses that cost around \$2000 [11]. Even for those who can, there are still issues such as limited battery life and lack of language support. This shows that the field still needs improvement. Through this project, we hope to take a step toward helping the community to make daily life easier for those who struggle with vision problems while also improving our skills in AI, real world project development, and teamwork.

1.5 Project Plan and Schedule

The following chapters detail the progression of the project, which was executed across two primary stages as visualized in the project schedule: Phase 1 (Research & Conceptual Design) and Phase 2 (Implementation & Integration). The initial phase, spanning the latter half of 2025, encompasses the first four chapters. Chapter One introduces the project scope, objectives, and the fundamental motivation for developing an assistive navigation tool. Chapter Two provides a literature review of modern assistive devices, identifying the limitations of current solutions like ultrasonic canes and commercial smart glasses. Chapter Three performs a deep problem analysis to specify technical requirements, while Chapter Four establishes the system's conceptual architecture, including the data flow between hardware and software components.

The second phase, conducted throughout the first half of 2026, focuses on technical execution and empirical evaluation. Chapter Five details the system implementation, covering hardware assembly on 3D-printed frames, model optimization for edge computing, and the integration of the Gemini 2.5 Flash API. Chapter Six presents the results of rigorous system testing, discussing the performance of the local safety loops and the cloud based cognitive loop in real world navigation scenarios. Finally, Chapter Seven offers concluding remarks on the project's success and outlines future work, such as the transition to dedicated AI accelerators and the miniaturization of the wearable hardware.

Figure 1.1 shows the Gantt chart for P this project. One can observe that most of the time is taken by the research about the topic in phase 1 and the software development in phase 2.

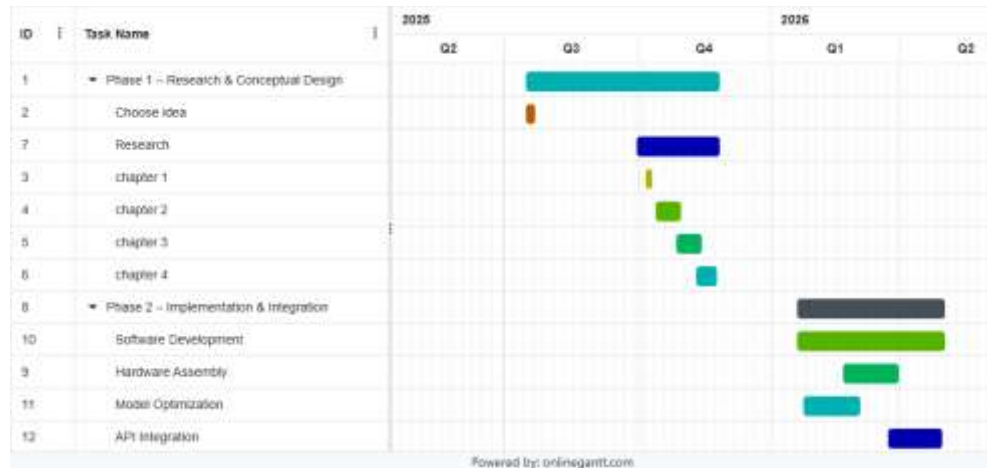


Figure: 1.1: Gantt Chart for the project

1.6 Outline of the Report

The following chapters will discuss the technologies that help people with visual disabilities, explaining how they interact with everyday tasks, and what limitations they still have.

We will also describe the components of our system, their functions and features, review existing related systems, compare them, and finally dive into the Problem Analysis.

CHAPTER TWO

2. LITERATURE REVIEW

2.1 Introduction

This chapter summarizes the current literature review and previous research on assistive technologies for the visually impaired. It shows how smart navigation systems have evolved from traditional tools comprised of white canes and guide dogs to modern AI based solutions that integrate sensors, cameras, and voice assistants. The aim of this review is to identify the strengths and weaknesses of the current approaches and set the basis for the proposed system.

In this chapter, we will talk about several assistive devices and systems, from ultrasonic based canes and camera equipped smart glasses to GPS navigation applications and AI powered sensor fusion systems. The chapter will explore how these technologies work and the potential limitations thereof, leading to the use case for the proposed AI powered smart glasses that integrate object detection, sensor based navigation, and a voice chatbot into one wearable solution



Figure 2.1: modern smart glasses designed for visually impaired users.

2.2 Background

Visual impairment remains one of the most challenging disabilities affecting daily human life. According to the World Health Organization (WHO), millions of people live with some form of blindness [9]. Most of them have face some difficulties in performing basic Things like walking around or finding their way in new places often prove difficult. These difficulties tend to make individuals rely on others for help and can affect the confidence, safety, and independence of those individuals.

Researchers and engineers keep working on ways to tackle these issues. They create technologies aimed at boosting mobility and sharpening awareness for those who are blind. Tools like white canes and guide dogs have offered solid support for years now. However, these tools have their own limitations. Take white canes, for instance. They only pick up on obstacles close enough to touch. Guide dogs demand a lot of training to get it right.

With recent advancements in Artificial Intelligence, computer vision, and embedded systems, it has become possible to develop better solutions [5]. This project builds upon these advancements by proposing a pair of AI powered smart glasses designed to help blind people walk safely and independently. The system will combine object detection models, distance sensors, sound navigation, and a voice chatbot, all integrated into a Raspberry Pi controlled platform to create a comprehensive assistive device.

2.2.1 Raspberry Pi Based System

The Raspberry Pi ends up being the core piece in this whole proposed project. It works as the main central processing unit (CPU) for everything involved. This setup stands apart from typical microcontrollers in a big way.

The Raspberry Pi counts as a strong single board computer (SBC) that stays affordable. It handles a complete Linux operating system without issues. That choice matters a lot for the design overall. It gives the right amount of processing strength to manage heavy jobs as they happen in real time.



Figure 2.2: Raspberry Pi 4 Model B, central processor for the proposed system.

The Raspberry Pi is capable of processing high resolution video streams from the camera and, executing complex object detection models such as YOLO (You Only Look Once) [1] or MobileNet SSD [2] using frameworks like TensorFlow Lite. This would not be feasible on a traditional microcontroller.

In this architecture, the Raspberry Pi will act as the brain. It will directly manage the camera module, run the AI inference, and also collect data from the sensors (e.g., ultrasonic sensors) connected via its General Purpose Input/Output (GPIO) pins. It will then fuse this information to understand the user's environment, which will trigger the appropriate feedback.

Its strong support for the Python programming language and libraries (like OpenCV for vision and gTTS for audio) makes it the ideal platform for the proposed project. Furthermore, its small form factor, low power consumption, and built in WiFi are necessary for a portable, wearable device and allow for future scalability.

2.2.2 Object Detection Models

The object detection models are one of the main features of the proposed project it is used to recognize and classify the environment. Object detection falls under computer vision. It lets computers identify various objects in pictures or videos. For this project, the camera will continuously capture the area nearby. Then the detection model goes through those images. Then it spots obstacles like walls, stairs, doors, or even vehicles. The project may use a lightweight AI models like YOLO or MobileNet SSD, which are optimized for real time detection and run efficiently on small computing devices. These models analyze the visual data and generate information about what objects are present [3], [6].

By merging it with distance measurements from sensors, it will create a more accurate understanding of the surroundings [4]. For instance, if the camera detects a “chair” and the ultrasonic sensor confirms it is within one meter, the system can issue a voice alert saying, " Careful, chair at 100 centimeters." This mechanism gives the user awareness that is a little bit similar to that of sighted individuals.



Figure 2.3: Output of an object detection model identifying multiple objects

2.2.3 Distance Sensors and Sound Navigation

In addition to object detection, the proposed project will need a distance sensor like an ultrasonic to detect nearby objects. They emit high frequency sound waves (ultrasound) that travel out, bounce off an object, and return as an echo. This method is particularly suitable for helping blind users, as it enables to detect obstacles in darkness or places where cameras might fail[3].

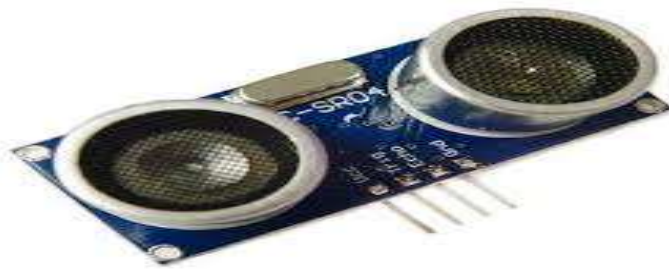


Figure 2.4: Ultrasonic Distance Sensor

The sensor readings will be processed by the Raspberry Pi and translated into sound navigation. The navigation system will use a voice alerts, to inform the user about the surroundings.

Moreover, directional audio can be used to indicate the position of obstacles on the left, right, or front side of the user. This intuitive guidance system helps users navigate safely without relying on a visual interface.

2.2.4 Hybrid Voice Interface

To make the proposed project reliable, the project will implement a hybrid (offline/online) voice interface. This design ensures that important features always work, regardless of internet connectivity.

Offline Mode: The core safety functions obstacle detection and object identification run entirely on the Raspberry Pi. Voice alerts are produced by a two tier text-to-speech system. Common alert phrases are pre rendered to audio using neural voices and cached on the device, so they play back

instantly and offline when a phrase is not yet cached, the system falls back to the `espeak-ng` engine, which synthesizes speech fully offline with no internet connection. Alerts combine the detected object with its distance, for example "Careful, chair at 50 centimeters," giving the user more useful information than a generic warning.

Online Mode: For more complex, interactive queries, the proposed project will provide a service when there is an internet connection. for example, When the user asks, “What’s in front of me?” or “Can you describe the room?” the device will:

1. Use a cloud based Speech-to-Text (STT) transcribe the user's voice.
2. Process this text to understand the user's needs
3. Generate a natural language response, which is then spoken back to the user.

This hybrid approach gives the best from both sides. It offers the solid reliability of an offline device. It also brings the advanced smarts of an online conversational assistant.

2.3 Existing Systems

In the last years, people have put a good effort into researching and development for helping devices. These devices aim to help those with disabilities. Different kinds of systems have come up over time. They rely on technologies like ultrasonic sensors, computer vision, GPS, and even AI driven speech tools. Looking at what already exists gives us some real clues about what works well and what falls short in today's options. It also points out areas where things could get better, and that's where this project fits in.

Here is a quick look at a few key systems that stand out. We will see how they stack up against the new AI powered smart glasses we have in mind

2.3.1 Ultrasonic Cane Systems

One thing people turn to a lot for folks who can't see is the ultrasonic smart cane. These canes come with sensors that pick up on obstacles pretty close by [6]. They give the user a heads up through vibrations or noises to help avoid trouble.



Figure 2.5: wewalk smart cane

For example, products such as the WeWALK Smart Cane use ultrasonic obstacle detection with smartphone connection [10]. The cane can detect obstacles that are within meters away and alert the user through vibrations. Some of the new versions also work with Google Maps helps on outdoor navigation.

While these systems are helpful, they have several limitations. The user must always hold the cane, which can be tiring and limits the user's ability to use their hands for another tasks. Also, smart canes typically detect obstacles in ground level only and will fail to identify objects at head height, such as signs or tree branches. The proposed project overcomes these issues by being hands free and offering upper body obstacle detection through cameras and sensors mounted on the glasses.

2.3.2 Camera Based Smart Glasses

Another category of assistive technology involves camera based smart glasses that use computer vision to analyze the surroundings. For Example, OrCam MyEye [11] and Aira Smart Glasses[12], both of which are

commercial devices designed to help visually impaired users identify objects, read text, and recognize faces.



Figure 2.6: OrCam MyEye

The OrCam MyEye For example, uses a small camera that is affixed to the frame of regular glasses. after recognizing printed text, barcodes, and even banknotes, it can read information to the user via an integrated speaker. By using an internet connection to link the user to a live agent, Aira adopts a different strategy. The glasses send a video stream to the agent, who offers verbal support in real time.

These systems, however, are often expensive and required an internet connection. The proposed project intends to provide similar levels of support in an affordable, offline capable, and fully automated manner using Raspberry Pi based hardware and locally running object detection models.

2.3.3 GPS and Mobile App Based Navigation Systems

Another group of related systems uses GPS technology and mobile apps to help blind users navigate. Apps like BlindSquare [13], Lazarillo [14], and Google Maps Accessibility [15] Mode assist visually impaired people in moving independently outdoors. These apps offer voice navigation, route planning, and location descriptions.

While GPS based systems work well for outdoor navigation, they struggle with indoors, where GPS signals are weak. They also fail to detect obstacles or provide awareness of the surroundings, which means users still need to depend on others.

In contrast, the proposed smart glasses combine local obstacle detection

using sensors and cameras with audio feedback. This ensures that both indoor and outdoor navigation can be done safely without depending only on GPS signals.

Table 2.1: Features and Limitations of different existing system

System Type	Main Features	Limitations
Ultrasonic cane	Detects nearby ground level obstacles	Requires use of hands, limited range, cannot detect upper obstacles
Smart glasses (e.g., OrCam)	Camera based recognition and voice output	Expensive, requires internet, limited obstacle detection
GPS mobile Apps	Voice based route guidance	Outdoor use only lacks real time obstacle detection

practical for everyday use from the comparison above, existing systems each address a specific part of the mobility problem but fail to provide a complete, integrated, and affordable solution. The proposed project aims to combine the advantages of all these systems real time object detection, sensor based distance measurement, audio feedback, and voice interaction into a single wearable device.

2.4 Contribution

The proposed project will be using computer vision, distance sensing, and a hybrid voice interface to facilitate the safe and independent mobility of

visually impaired people. Unlike existing solutions that are either fully online or fully offline, this proposed project will be a hybrid system. This design helps us to make safety features always available offline, and advanced conversational features when there is an internet connection.

Key contributions include:

- Real time Offline Object Detection: Could be developed using AI models, such as YOLO or MobileNet SSD, to detect objects and obstacles that are in the surroundings.
- Hybrid Audio Based Navigation: Provides critical, real time voice alerts (e.g., "Careful, chair at 50 centimeters") using an offline TTS engine, with more detailed environmental descriptions available when connected to the cloud.
- Online Voice Chatbot Integration: This will enable natural voice communication between the user and the system for advanced queries, enhancing ease of interaction and accessibility.
- Low cost and wearable system: Using a Raspberry Pi and modular components to ensure affordability, scalability, and comfort during daily use.

This combination of AI driven perception, sensor fusion, and a hybrid voice interface makes the proposed system unique among assistive technologies. The primary contribution is to provide reliable navigation that helps visually impaired people by always describing their surroundings (offline) and, when connected, allowing them to query their environment using a natural voice chatbot.

CHAPTER THREE

3. PROBLEM ANALYSIS

3.1 Introduction

This chapter picks up right where the literature review left off. It shifts things from those abstract ideas into real world uses. The section lays out the main problems the project wants to tackle. It digs into what causes them at the core. Then it breaks down how the system works overall. Along the way, it spots the key functional needs and the non-functional ones too. Looking closely at all this stuff sets up a solid ground for putting together solutions that actually work. That way, the whole design lines up with what the project set out to do

3.2 Problem Specification

To successfully address the challenges outlined in Chapter 1, the proposed system must solve several distinct technical problems. Simply providing a general solution is insufficient; the system's success depends on meeting specific, measurable goals.

The core problem is one of real time sensor fusion and data interpretation. The system must effectively answer three questions for the user, all at once:

1. "What is it?" (Object Recognition): The system must accurately identify and classify key objects in the user's path (e.g., 'person', 'chair', 'stairs') from a live video stream. This requires a fast and efficient computer vision model.
2. "Where is it?" (Obstacle Detection): The system must detect the

presence and distance of all obstacles, including those that are hard to see (like glass doors) or are outside the camera's view (like a low lying object).

3. "How do I interact?" (User Interface): The system must communicate this complex information to the user in an intuitive, non-overwhelming, and timely manner via audio. It must also be able to understand and process the user's natural language queries.

Therefore, this project must solve the specific challenge of integrating these three components vision, sensing, and interaction into a single, low power, and wearable device that functions reliably in unpredictable environments. The following sections will define the precise requirements needed to solve this problem.

3.3 System Analysis

System Analysis helps the proposed project's technical implementation through a comprehensive study of the necessary components and requirements. This includes a careful review of existing solutions and a focused effort to accurately define what does proposed project must achieve to be successful. This analysis is considered as a key factor to determine both the functional requirements (what a system should do) and non-functional requirements (how well the system should perform its functions). By collecting and analyzing these requirements, the project will ensure that the final design is practical, realistic, and aligned with the goal of our proposed project.

3.3.1 Requirement Collection

The requirements for this proposed project have been collected using three different approaches.

First, by studying existing systems like Smart canes, AI based glasses, and GPS guided mobility tools, these systems provided a clear understanding of their functionality and limitations. These analyses help find what the missing features are and helps us to clarify the functional goal of this proposed project.

Second, we did some brainstorming workshops; these Sessions help the group to generate new ideas, refine objectives, and turn observations into technical needs. Through discussions, key capabilities were identified, such as offline alerts, object detection, and voice interaction, based on feasibility and potential impact.

Finally, we reviewed several research papers, academic sources, and technology reports related to assistive devices for visually impaired users. This review validate the requirements identified during brainstorming sessions and ensured the system's alignment with current computer vision, embedded computing, and accessibility technologies.

Together, these methods ensure that the system was based on real world needs, based on existing solutions, and supported by academic research.

3.3.2 Requirement Analysis

These collected requirements have been analyzed in detail to ensure that the proposed project will be capable of meeting its objectives and operating reliably in real conditions. The analysis is grouped into a functional and non-functional requirements.

1. Functional Requirements

These are the requirements that prescribe what the system shall do to support visually impaired users:

1. **Real Time Object Detection:** This system needs to be able to detect and classify common objects, such as people, doors, or stairs, from a live camera feed using lightweight AI models like MobileNet SSD or YOLO Lite.
2. **Obstacle Distance Measurement:** Ultrasonic sensors have to precisely measure the distance of obstacles in proximity and send this data for processing to the Raspberry Pi.
3. **Audio Feedback Generation:** The system needs to transform detection results into speech warnings in offline mode or detailed descriptions in online mode.
4. **Hybrid Voice Interaction:** It needs to understand user and respond through offline or online chatbot capabilities when internet access is available.
5. **Sensor Fusion Processing:** This depends on a combination of data from the camera and ultrasonic sensors to give correct and timely feedback to its user.

2. Non-Functional Requirements

These requirements specify how the system is to perform its tasks:

1. **Real Time Performance:** End-to-end detection and audio feedback must occur within an acceptable latency.
2. **Accuracy:** Object detection models need to attain reliable accuracy to avoid cases of false positives or missed obstacles.
3. **Portability and Comfort:** All components must be compact, lightweight, and suitable for wearable use.
4. **Energy Efficiency:** The system must operate for extended periods using portable power without overheating or consuming excessive energy.
5. **Reliability:** Offline safety features (object and obstacle detection)

must always work, regardless of internet availability.

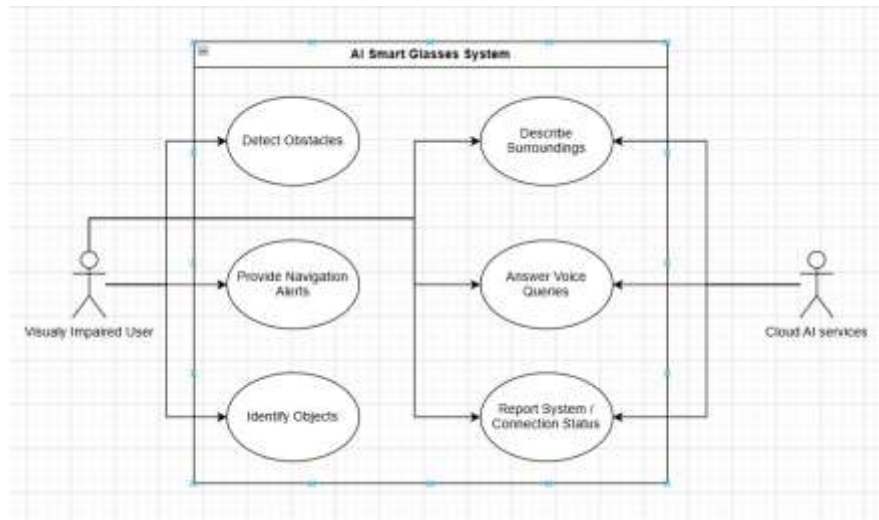


Figure 3.1: Use Case Diagram for the AI Smart Glasses System

Table 3.1: Functional vs. Non-Functional Requirements of the System

Aspect	Functional requirements	Non-functional requirements
Purpose	Defines what the system must do	Defines how well the system must perform
1	Real Time Object Detection	Real Time Performance
2	Obstacle Distance Measurement	Accuracy
3	Audio Feedback Generation	Portability and Comfort

4	Hybrid Voice Interaction	Energy Efficiency
5	Sensor Fusion Processing	Reliability

3.4 Implementation and Evaluation Plan

This section lays out the full plan for Phase 2. It centers on moving from the conceptual design in Phase 1 over to a working prototype that can be tested. The way we implement things takes a modular and iterative path. We start by integrating the hardware and software parts. Then we build out the separate software pieces one by one. All that leads up to pulling everything together into one complete system. After the plan itself comes a solid method for evaluating how well it all works. That evaluation looks at performance in both numbers and more general ways. It checks everything against the functional and non-functional needs we spotted earlier in the chapter.

3.4.1 Technical Tools and Programming Languages

What The selection of technical tools and programming languages is an important thing to make our proposed system successful.

- Python will serve as the primary programming language for this proposed project. It provides extensive libraries for artificial intelligence (TensorFlow), computer vision (OpenCV), and hardware interfacing (gpiozero), which makes it perfect for us. Its readability and rapid development capabilities are ideal for prototyping and integrating complex modules.
- Bash Scripting Basic shell scripts will be used for automating system startup and management tasks on the Raspberry Pi OS, such as launching the main Python application and managing system

services.

3.4.2 Frameworks

Frameworks provide pre built functionality to help us implement the proposed project's features.

- TensorFlow Lite As the project requires running complex object detection models like MobileNet SSD or YOLO. The TensorFlow Lite (TFLite) will be the primary deep learning framework. TFLite offers optimized performance (low latency, small binary size) which makes it perfect for a real time wearable device.
- OpenCV (Open Source Computer Vision Library) will be used for all real time video processing tasks. OpenCV functions are important to get images from the camera module, also perform pre-processing on each frame before feeding it to the TFLite model.
- gpiozero This is a Python library, and it's important for interfacing with the sensors. It will be used for controlling the General Input/Output (GPIO) pins on the Raspberry Pi to trigger the sensors and read their distance measurements.
- Cloud Speech-to-Text (STT) API (e.g., Google Cloud STT) is great for the Online Mode of the hybrid voice interface, a cloud based STT API will be used to transcribe the user's spoken queries. This approach will provide high accuracy.
- TTS engine will be used as the Text-to-Speech (TTS) engine. Its ability to run without Wi-Fi makes it the best choice for delivering essential safety alerts (e.g., "Obstacle, one meter") without internet dependency.
- Gemini 2.5 flash this Unlike traditional object detectors that rely on predefined classes, this API provides Multimodal Scene Understanding, allowing the system to interpret complex visual data and generate natural language descriptions, a.

3.4.3 Tools

This part points out the exact hardware and software tools needed to put together the prototype.

1. Hardware Tools:

- Central Processor: Raspberry Pi, selected because it strikes a good balance with processing power that can handle TFLite models. Plus it uses low power.
- Vision Sensor: Camera or something like a compatible USB webcam. That setup captures the video feed just fine.
- Distance Sensors: involve ultrasonic types. They pick up obstacles and measure how far away things are. This helps make up for what the camera might miss.
- Audio I/O: microphone along with a small lightweight speaker. Those take care of the voice interface side of things.

2. Software Development Tools:

- Operating System: Raspberry Pi OS. It is a Debian based Linux setup that works well with the hardware.
- IDE: Visual Studio Code. It includes the Remote SSH extension. That lets you develop code smoothly on your main computer. Then you deploy and debug right on the Raspberry Pi.
- Version Control: Git and GitHub. They manage the source code. You track changes that way. It also makes team collaboration easier.

3.4.4 Evaluation Methodology

To check if the system works well overall, we will carry out a multi stage evaluation plan. This plan will look at each part separately and then the

whole system together.

1. Component Level Performance:

- Object Detection Model:
 - Accuracy: The model we pick, like MobileNet SSD, will face standard computer vision checks. We will use Mean Average Precision, or mAP, on a test dataset set aside for common objects in the environment.
 - Speed: We will clock it in Frames Per Second on the Raspberry Pi 4. The goal is to make sure it handles real time needs without issues.
- Distance Sensor:
 - Accuracy: We will compare its readings to a tape measure at different ranges, such as 0.5 meters, 1 meter, and 3 meters. From there, we can figure out the error margin.
- Voice Interface:
 - Offline TTS: we will measure latency. That means timing from the text command to when the audio plays out.
 - Online STT: accuracy gets measured with Word Error Rate, or WER. We will run it on a set list of user commands that we define ahead.

2. System Level Integration Testing:

- End-to-End Latency: the main measure for the core function. We will track the full time from when an object shows up in the camera view until the descriptive audio alert comes through. This timing proves vital for keeping users safe.
- Hybrid Mode Switching: The system needs to shift smoothly between offline safety alerts and online chatbot features. We will do this by connecting and then cutting off the Wi-Fi on the device.

3. Qualitative User Acceptance Testing (UAT):

- Scenario Based Testing: we will set up controlled trials to mimic real situations. Examples include navigating a hallway, identifying a chair, or asking what lies in front of the user.
- User Feedback: (If feasible and ethically approved) Test subjects, or team members simulating user behavior, will provide qualitative feedback on:
 - Effectiveness: Did the device provide accurate and timely information?
 - Intuitiveness: Was the audio feedback clear and easy to understand, or was it overwhelming?
 - Usability: Is the device comfortable and non intrusive for its intended use?

CHAPTER FOUR

4. SYSTEM DESIGN

4.1 Introduction

This chapter presents the design of a proposed intelligent assistance solution system, progressing from theoretical understanding to practical application. It describes the system's architecture, interactions, and how its main components work together to provide real time navigation for visually impaired users. The chapter begins with the system design specifications, which define the main units and their roles. The system layout is then presented, accompanied by diagrams, data flow, and overall system behavior. To clarify the logic of the proposed project, when these components are combined, they provide a precise and comprehensive blueprint that guides the proposed project development and ensures alignment with its objectives.

4.2 System Design Specification

The system design specification defines the major modules that make up the smart glasses with AI powered visual assistance and describes how each part will contribute towards achieving their key objective of real time navigation assistance for visually impaired users. These five central units are the processing unit, vision module, sensing module, audio interface, and communication module. Each unit is critical in converting raw data on the environment into meaningful feedback for the user.

1. Processing Unit (Raspberry Pi 4 Model B) The main control unit, Raspberry Pi, runs AI models, input from the camera, processing

data from sensors, and system control in general. It runs an object detection model using TensorFlow Lite, fuses sensor outputs, and sends results to the audio subsystem for presentation. Due to its lightweight size and low power operation, it is fit for a wearable environment.

2. Vision Module (Camera Sensor) The camera continuously captures real time images from the user's surroundings. These images are fed into the object detection model, which will be described next. Common objects such as people, stairs, and obstacles become input into this model. Situational awareness and recognizing objects need high resolution visual input from the vision module.
3. Distance Sensing Module (Ultrasonic Sensors) These are ultrasonic sensors detect the distance of obstacles within their range by emitting and receiving high frequency sound waves. This module, therefore, provides depth information that is not captured by cameras, especially under low light conditions or when transparent objects such as glass doors need to be detected. The distances are fused with the detection results using the Raspberry Pi for improved accuracy.
4. Audio Output Module (Speaker & TTS Engine): This module delivers real time voice feedback. It uses a layered text-to-speech approach high quality neural voice clips are pre cached on the device for the recurring safety alerts, while the espeak-ng engine provides an always available offline fallback and handles dynamic responses (such as the assistant's scene descriptions). Essential safety alerts therefore remain available without any internet connection, and richer descriptions are spoken when an online query is made.
5. Language Module: The system supports both English and Arabic. At startup the user selects a language by voice, and that choice then drives every spoken interface obstacle alerts, the assistant's responses, and the speech recognition locale.
6. Voice Input & Interaction Module: Microphone + STT System The

user voice commands are captured by a microphone. The system works in two modes:

- Offline mode: deal with simple commands, answers with its inbuilt logic.
 - Online Mode: This relays the user query to a cloud based chatbot, which then returns responses that sound natural. The hybrid design means it's reliable and provides advanced interaction when available.
7. Power Supply Module (Portable Battery Pack) A compact power bank provides the required voltage and ensures mobility. The system is optimized for energy efficiency to support extended usage.
 8. Integration & System Control Software The Raspberry Pi runs custom Python scripts that manage:
 - Frame capture and pre-processing
 - Object detection inference
 - Sensor fusion logic
 - Voice command recognition

Audio output generation Switching mode between offline and online operation This software, acting as the "brain logic," makes sure that all modules work in concert and provide accurate and timely feedback.

4.3 Design Architecture

This section provides the complete architectural design of the proposed project. It describes how the system processes visual and sensor data in parallel to maintain user safety while offering high level environmental understanding. Two diagrams are included to represent the system's operational workflow and the hardware software architecture that enables real time assistance.

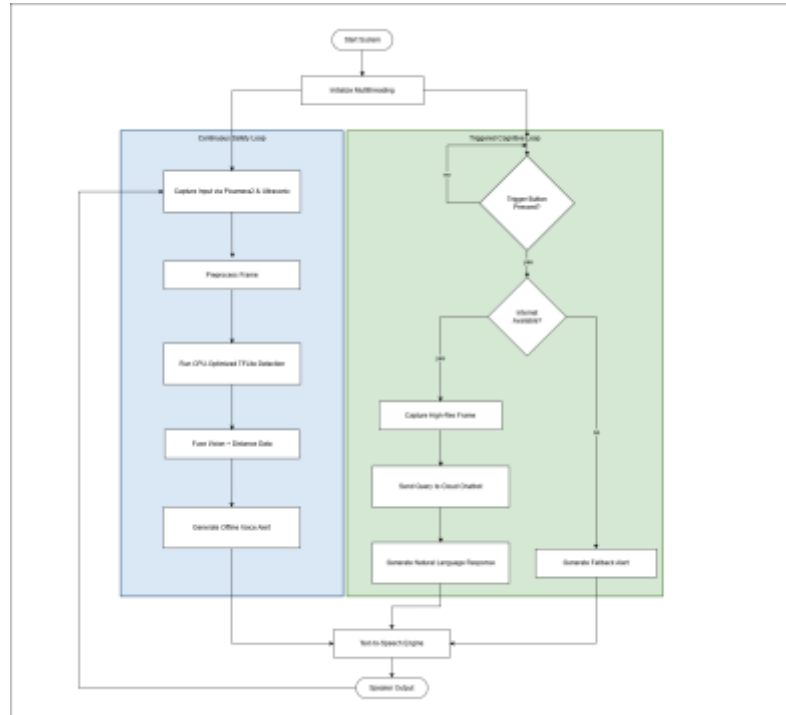


Figure 4.1: System flowchart for the AI Smart Glasses System

The flowchart demonstrates the end-to-end process used by the system to analyze the environment through a dual loop multithreaded architecture. Upon system startup, the software initializes multithreading to separate safety critical tasks from on demand cognitive queries. This parallel structure ensures that the system can think on two levels: a fast, instinctive level for safety and a deeper, more analytical level for scene understanding.

1. **Continuous Safety Loop (Offline):** This high priority loop runs continuously to ensure immediate hazard detection. It captures input via the Picamera2 and ultrasonic sensors, performs frame preprocessing, and runs CPU optimized TFLite detection locally. By fusing vision and distance data, the system generates instantaneous offline voice alerts to inform the user of immediate obstacles.
2. **Triggered Cognitive Loop (Hybrid):** This loop remains idle until a physical trigger button is pressed. Once activated, the system checks for internet availability. If online, it captures a high resolution frame and sends a query to the cloud chatbot, Gemini 2.5 Flash, to generate

a detailed natural language response. If the internet is unavailable, a fallback alert is generated to maintain system reliability.

Both loops eventually converge at the Text-to-Speech (TTS) engine, which manages the audio queue to provide clear, non overlapping feedback through the speaker output. This dual track workflow ensures that safety critical distance checks are never blocked by the processing time or network latency of complex cloud based queries.

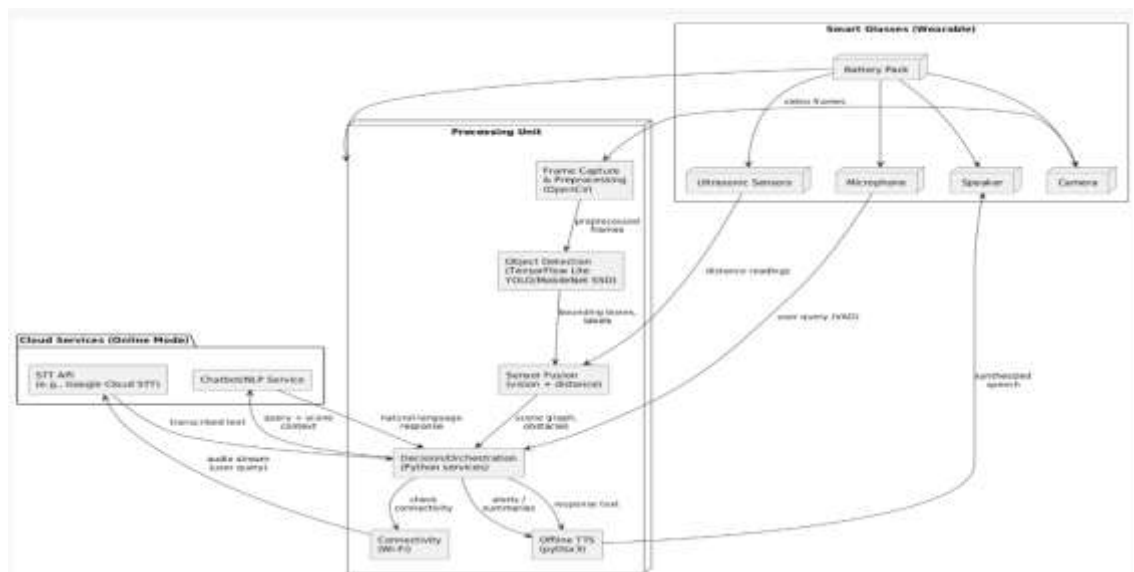


Figure 4.2: System architecture for the AI Smart Glasses System

The system architecture diagram shows how the hardware and software components of the smart glasses work together to provide real time assistance. The camera, microphone, and ultrasonic sensors collect visual, audio, and distance data, which are sent to the processing unit. Inside the processing unit, frames are preprocessed, passed through the object detection model, and combined with distance readings using sensor fusion to produce an accurate understanding of the environment.

The decision module determines the type of response needed and checks whether an internet connection is available. If online, the system uses cloud based services for speech recognition and natural language responses. If offline, it generates essential alerts using a local Text-to-Speech engine.

Finally, the processed information is delivered back to the user through the speaker, completing the feedback loop. This architecture ensures reliable operation, supports both offline and online modes, and provides continuous awareness for visually impaired users.

CHAPTER FIVE

5. SYSTEM IMPLEMENTATION & INTEGRATION

5.1 Introduction

The implementation phase involves the technical realization of the proposed project, transitioning from theoretical design to a functional wearable system. This chapter details the integration of edge computing, localized object detection, and high level multimodal AI.

The primary engineering focus was the synchronization of a real time Safety Loop (ultrasonic sensing and local inference) with a high latency Cognitive Loop (cloud based assistant). The implementation was structured into three core areas: the physical assembly of the hardware on 3D-printed frame, the optimization of the model for edge execution, and the development of a multithreaded software architecture to ensure concurrent processing. By modularizing these components, the proposed project achieves a balance between immediate hazard detection and complex environmental understanding.

5.2 Implementation and Evaluation Plan

Moving the proposed project from a controlled breadboard environment to a wearable, 3D-printed frame required a multi layered implementation strategy. The goal was not just to make the components fit together, but to ensure they worked in harmony under real world conditions where factors like physical movement, battery life, and heat management become critical.

5.2.1 Hardware Integration and Wearable Assembly

Bringing the project to life meant moving from a breadboard prototype to a wearable build and building something a person could actually wear. The physical build was designed around a modified Wayfarer frame, which offered enough room to hide the electronics without being too bulky or uncomfortable for the user. The goal was to make the technology feel like a natural extension of the person wearing it.

The following diagram shows the internal wiring used to link the brain of the system the Raspberry Pi to the sensors and the physical buttons that make the device work:

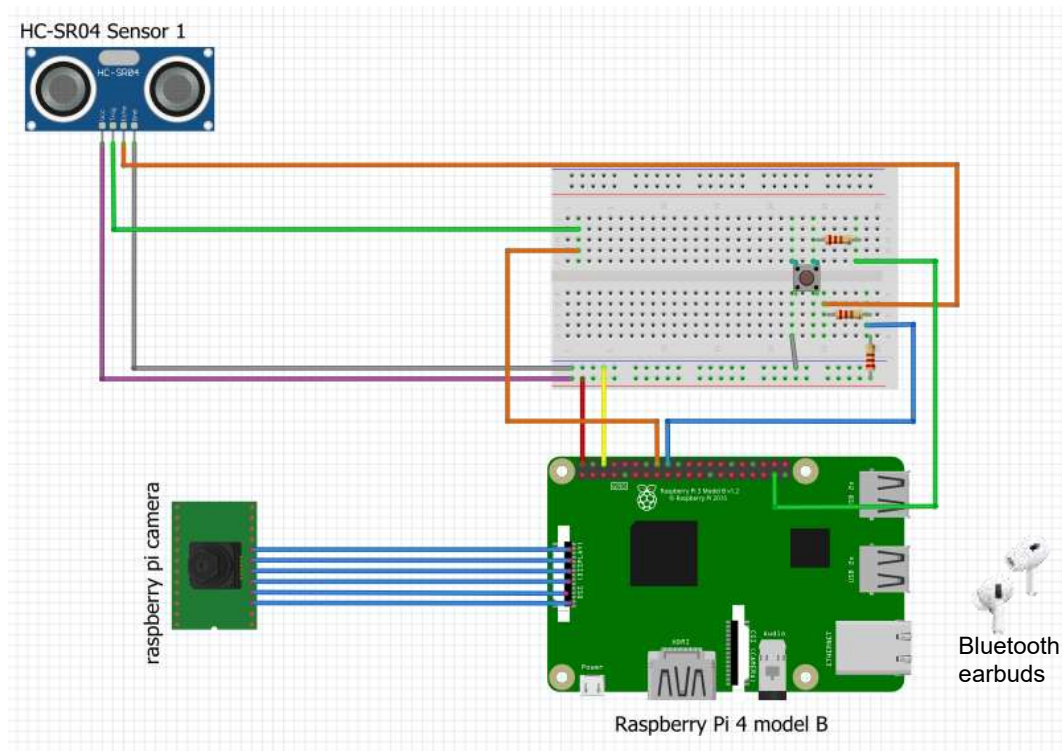


Figure 5.1: Hardware wiring and component interconnection schema

The assembly was guided by these specific hardware choices:

- Where the Sensors Sit: We placed the ultrasonic sensor right at the bridge of the glasses. This gives it a clear, forward looking view that matches where the user is looking, which is vital for getting accurate distance readings on obstacles.
- Wiring and Clean Design: To keep the glasses looking like actual

glasses, we tucked fine gauge wiring inside the frame arms. We used GPIO pins 23, 24, and 26 to handle the hardware interrupts, specifically so the tactile button for the AI assistant would feel snappy and responsive.

- **Untethered Audio:** Instead of having wires hanging down to the user's ears, we integrated Bluetooth earbuds. This keeps the user's hands free and ensures they can hear both the urgent distance beeps and the detailed AI descriptions clearly and privately.
- **Keeping it Powered:** The Raspberry Pi 4B is power intensive, especially when it's processing video and sensor data simultaneously at the same time. To handle this, we used a portable 5V/3A power pack to keep everything running smoothly without the system slowing down or crashing.

5.2.2 Software Environment and Model Optimization

The software side of the implementation focused on making the most of the Pi's limited resources.

- **Local Intelligence:** We deployed SSD MobileNet because it strikes a perfect balance between speed and accuracy. Rather than sending every frame to the cloud, the project handles basic object detection locally, which keeps the system responsive and preserves battery life.
- **Concurrent Architecture:** A major part of the implementation was setting up a multithreaded environment. We designed the software so that the safety critical distance checks never have to "wait" for the more complex AI assistant to respond. This ensures that even if the internet lags, the basic obstacle detection remains functional.

5.2.3 Implementation Objectives

The final stage of the implementation plan focused on verifying the

integration of the hardware and software modules. Rather than analyzing the final performance data which is detailed in the following chapter this phase was dedicated to ensuring system stability. The priority was to confirm that the "Safety Loop" and "Cognitive Loop" could run simultaneously without software conflicts, and that the 3D-printed housing provided adequate protection for the internal circuitry during mobile use. This verification served as the final "green light" before moving into formal field testing.

5.2.4 Audio Output and Neural Voice Caching

Because the device communicates entirely through sound, the quality and timing of speech output were treated as a core design problem rather than an afterthought. A naive approach would synthesize every alert on demand, but on device synthesis is either slow or robotic, and a visually impaired user relies on these alerts being both immediate and clear. To resolve this tension, the audio module was implemented as a two tier text-to-speech system that balances quality against guaranteed availability.

The primary tier is a pre rendered neural voice cache. Every alert the system can produce is a template of the form "Careful, [object] at [distance] centimeters," and the set of distances is deliberately bounded by floor bucketing each reading to the nearest ten centimeters. This makes the space of possible phrases finite, so each one can be synthesized once using a high quality neural voice (Microsoft Edge en-US-GuyNeural for English and ar-SA-HamedNeural for Arabic) and stored as an audio file on the device. At runtime, the system simply plays the matching cached file, achieving natural sounding speech at roughly eighty milliseconds of latency without re synthesizing anything.

The second tier is the `espeak-ng` engine, which guarantees the system is never silent. If a required phrase has not yet been cached, `espeak-ng` speaks it immediately and fully offline, while a background thread synthesizes the

missing neural clip and writes it to the cache for future use; the write is performed atomically so a partially generated file is never played. The same fallback engine also handles speech that cannot be pre cached, most notably the variable scene descriptions returned by the assistant. All playback is serialized through a lock, and any alert that arrives while audio is already playing is skipped rather than queued, which prevents the overlapping or stale announcements that would otherwise confuse the user.

It should be noted that generating a neural clip for the first time requires an internet connection, since synthesis is performed by a cloud service. Once a clip is cached, however, its playback is fully offline, and `espeak-ng` provides an unconditional offline guarantee in every case. The safety critical audio path is therefore always available regardless of connectivity, with neural quality output provided opportunistically whenever a phrase has been cached.

5.3 Model Implementation

The object detection capability of the system was developed in two stages. The first stage involved training a custom detector to establish the accuracy achievable on the project's target classes, and the second stage involved selecting a model architecture that could meet the real time constraints of the wearable hardware. This two stage approach made it possible to separate the question of whether the detection task was learnable from the question of whether it could be run fast enough on a resource constrained device.

5.3.1 Dataset and Training

A custom object detector was trained on a purpose built dataset of 32,511 annotated images containing 54,480 object instances, assembled to cover the objects most relevant to pedestrian navigation for a visually impaired user. The dataset's raw categories were consolidated into 24 final classes including person, car, bicycle, bus, stairs, door, crosswalk, traffic cone,

pothole, and the three traffic light states by merging duplicate labels during a conversion from COCO to YOLO annotation format. After conversion, the data was split into 31,944 training images and 4,189 validation images.

The detector was based on the YOLOv11-Nano architecture, the smallest variant in the YOLOv11 family, chosen because its low parameter count makes it the most plausible YOLO candidate for embedded deployment. The model was trained for 80 epochs at an input resolution of 320×320 pixels. This resolution was selected deliberately it matches the native scale of the dataset imagery and substantially reduces the computational cost of inference, which is critical for any prospect of running the model on the Raspberry Pi. An early stopping criterion was applied to halt training once validation performance plateaued, preventing overfitting[8].

5.3.2 Training Results

On the validation set, the trained model achieved a mean Average Precision (mAP@0.5) of 0.713 and an mAP@0.5:0.95 of 0.494, with an overall precision of 0.82 and recall of 0.67. These results confirmed that the detection task was well posed and that the chosen classes were learnable to a useful standard. Performance was strongest on visually distinctive, safety critical classes stop signs (mAP@0.5 = 0.98), doors (0.98), fire hydrants (0.98), crosswalks (0.95), and trees (0.94) all exceeded 0.94 which is encouraging given that these are precisely the landmarks most useful for navigation. Accuracy was weaker on a small number of classes, this is attributable to high intra class variation and, for some classes, a relatively small number of training instances. The training results is shown in the Figure 5.2

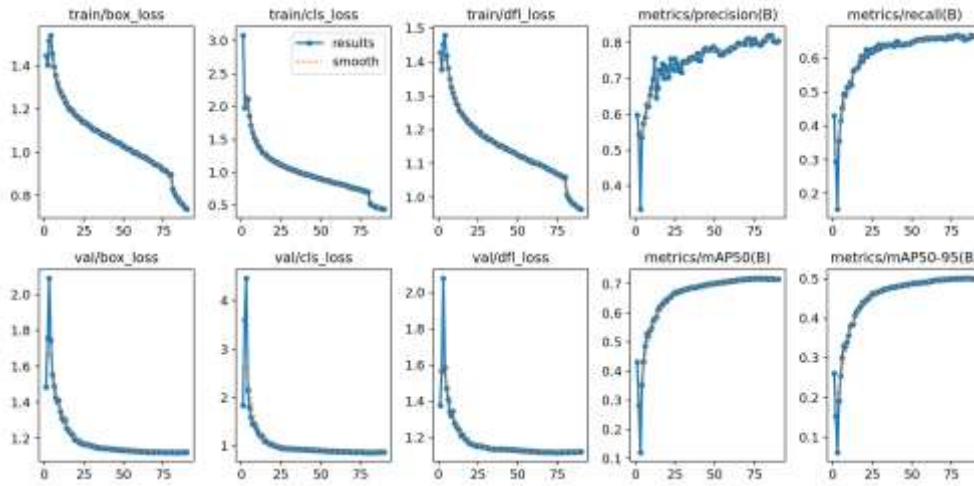


Figure 5.2: training result of yolo model

5.3.3 Model Selection for Deployment

Although the trained YOLOv11-Nano model achieved strong accuracy, accuracy alone does not determine suitability for a wearable device. The decisive constraint is inference speed on the Raspberry Pi 4B, which has no dedicated GPU or neural accelerator, because a delayed obstacle warning is not merely inconvenient but a direct safety risk to the user. This concern is supported by published benchmarking of object detection models on edge hardware: on the Raspberry Pi 4 without an accelerator, YOLOv8 inference has been measured at up to roughly 3.7 seconds per image, whereas an SSD-MobileNet model completed inference in approximately 200 milliseconds on the same device more than an order of magnitude faster at the cost of lower detection accuracy [7].

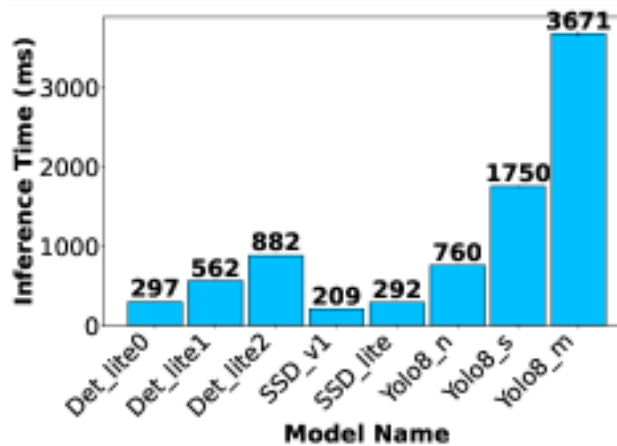


Figure 5.3: comparison between different models running on raspberry pi 4[7]

Given this trade off, the system adopted SSD-MobileNet as the deployed detector for the real time Safety Loop, running through the TensorFlow Lite interpreter. SSD-MobileNet is explicitly designed for efficient inference on resource constrained devices [2] and is well suited to optimized edge deployment [16], allowing the Safety Loop to sustain continuous real time detection while leaving sufficient CPU headroom for the concurrent audio and ultrasonic sensor threads. The accuracy reduction relative to YOLO was judged acceptable for this role, since the Safety Loop's purpose is the reliable, low latency detection of common obstacles rather than fine grained classification the richer, slower contextual understanding is instead delegated to the cloud based Cognitive Loop. The trained YOLOv11-Nano model was retained as a documented baseline and was exported to several edge optimized formats (NCNN, OpenVINO, and INT8 TFLite) during evaluation. Figure 5.3 contrasts the two architectures, pairing the on device speed gap reported in [7] and illustrates why the faster SSD-MobileNet was selected for deployment despite YOLO's higher precision.

5.4 Software Development and Algorithms

The heart of the proposed project lies in its ability to process environmental data in real time. Developing the software required more than just writing functional code; it required creating a system that could think on two levels: a fast, instinctive level for safety and a deeper, more analytical level for scene understanding.

5.4.1 The Multithreaded Execution Model

One of the most significant hurdles during development was preventing "blocking." In a standard sequential program, if the system is waiting for a response from a cloud API, everything else stops. For a visually impaired user, a two second delay in a distance sensor could be dangerous.

To solve this, we implemented a multithreaded architecture using Python's

threading module.

- The Safety Thread: This is a high priority loop that polls the ultrasonic sensor and runs the local object detector. It operates at a consistent frequency, ensuring that the distance based audio pings are never interrupted.
- The Cognitive Thread: This thread remains idle until the user presses the trigger button. When activated, it captures a high resolution frame and handles the communication with the Gemini 2.5 Flash API. By isolating this in its own thread, the heavy network traffic and processing time of the AI assistant don't interfere with the real time proximity alerts.

5.4.2 Local Detection and the SSD MobileNet Pipeline

For the localized vision component, we chose the SSD MobileNet model. While newer and larger models exist, they often run too slowly on edge hardware like the Raspberry Pi.

- Efficiency by Design: MobileNet uses depthwise separable convolutions, which essentially break down a heavy mathematical process into smaller, manageable pieces. This allowed the proposed project to maintain a usable frame rate for identifying common obstacles like chairs, tables, or people without the need for an active internet connection.
- Optimization: We utilized the TensorFlow Lite interpreter to load the pre trained weights, focusing on a specific set of 26 classes relevant to indoor and urban navigation. This kept the system "lean," ensuring that the CPU had enough overhead to handle the audio synthesis concurrently [16], [2].

5.4.3 Semantic Intelligence and API Integration

While local detection is great for identifying "a chair," it cannot describe the context, such as "a wooden chair with a jacket hanging on it." This is where the integration with Gemini 2.5 Flash comes in.

- **The Trigger Logic:** We developed a specific handler for the GPIO pin 26. A single press triggers a "Vision to Language" request. The software captures a frame, encodes it, and sends it to the cloud with a prompt tailored for assistive navigation.
- **Natural Language Output:** The final step of the algorithm is converting these AI insights into speech. We focused on ensuring the audio feedback was clear and concise, prioritizing immediate safety information over long, descriptive sentences to avoid overwhelming the user's auditory senses.

5.5 System Integration and Orchestration

The final phase of implementation involved merging the independent hardware and software modules into a single, cohesive unit. This orchestration was the most delicate part of the project, as it required the high speed electronic signals from the sensors to align perfectly with the processing cycles of the AI models.

To achieve this, we developed a centralized "Main" controller script that serves as the system's brain. This script is responsible for initializing the GPIO pins, loading the local detection models into memory, and establishing a secure connection to the Gemini API. We focused heavily on the timing of the data loop if the ultrasonic sensor polled too slowly, the user might miss a hazard, but if it polled too fast, it could overwhelm the Raspberry Pi's CPU and cause the vision script to lag. Finding this balance was essential for making the device feel responsive and reliable in a mobile environment.

5.6 Code Structure and Repository Organization

To ensure the project remains maintainable and scalable, the software was developed using modular, class based architecture. Rather than a single, monolithic script, the code is divided into specific modules, each responsible for a distinct part of the system's "senses."

1. `main.py` (The Orchestrator): This is the entry point of the application. It handles the high level multithreading logic, initializing the different sensors and ensuring that the data flows correctly between the vision and hardware modules.
2. `hardware.py` (The Physical Layer): This module encapsulates all direct interactions with the Raspberry Pi's GPIO pins. It contains the logic for polling the ultrasonic sensor and setting up the interrupt handlers for the tactile button, ensuring that the hardware remains decoupled from the AI logic.
3. `vision.py` (The Perception Layer): This script manages the local SSD MobileNet pipeline. It handles the camera stream, processes each frame for object detection, and returns the class labels and bounding boxes to the main controller.
4. `assistant.py` (The Cognitive Layer): Dedicated to the cloud based intelligence, this module manages the connection to the Gemini 2.5 Flash API. It handles image encoding and the formatting of the natural language prompts that are sent to the AI for scene description.
5. `audio.py`: A utility script that manages the text-to-speech engine, ensuring that alerts and AI responses are queued and played without overlapping or crashing.

By organizing the repository this way, we ensured that a bug in the audio script wouldn't crash the safety critical distance sensors, and it allowed for easier debugging of individual components during the integration phase.

5.7 Challenges and Problem Solving

Transitioning the VIAD system from a conceptual design to a functional software prototype required overcoming several computational hurdles related to real time processing and resource management.

1. Multithreaded Resource Management

Running two parallel AI pipelines on the Raspberry Pi 4B's CPU created significant resource contention. The "Cognitive Loop" required heavy network I/O and image encoding for Gemini 2.5 Flash API requests, which threatened to delay the safety critical "Safety Loop". To resolve this, we implemented an Asynchronous Priority System using Python's threading module, ensuring that the safety thread for ultrasonic polling and TFLite detection remained deterministic and never "waited" for cloud responses.

2. Model Selection and Inference Optimization

Initially, we trained a custom YOLOv11-Nano model to handle specific navigation hazards. However, during integration testing, we determined that the YOLO architecture was too computationally expensive for the Raspberry Pi's CPU, leading to unacceptable frame rate lag. To achieve real time performance, we switched to an SSD MobileNet model loaded via the TensorFlow Lite interpreter. By optimizing the preprocessing pipeline and limiting the detection scope to 26 relevant classes, we maintained a consistent frame rate suitable for safe navigation.

3. Exception Handling for Hybrid Logic

Maintaining system stability during network fluctuations was a critical safety requirement. Unstable internet connections could cause API calls to hang, potentially freezing the entire application. We implemented a Non

Blocking Fallback Mechanism with strict timeout handlers. If the cloud query fails to respond within a set window, the software bypasses the request and triggers a local fallback alert, ensuring the user is never left without feedback.

CHAPTER SIX

6. SYSTEM TESTING AND RESULT DISCUSSION

6.1 Introduction

Introducing The evaluation phase of the proposed project was designed to move beyond basic code verification and into the realm of real world reliability. For an assistive device intended to aid the visually impaired, the "success" of the system cannot be measured solely by whether the software runs; it must be measured by the precision of its environmental awareness and the speed of its feedback. This chapter presents a critical analysis of the system's performance, focusing on how the integrated hardware and software components behave under various operational stresses.

The testing strategy was divided into three primary objectives:

1. Safety Reliability: Assessing the accuracy of the local detection model and the ultrasonic sensor in identifying immediate hazards.
2. System Latency: Measuring the time delay between environmental changes and the delivery of audio feedback to the user.
3. Interaction Quality: Evaluating the descriptive depth provided by the cloud based AI assistant when faced with complex, non standard scenes.

By subjecting the proposed project to these metrics, we can identify the practical boundaries of the current prototype and discuss the trade offs made between processing speed and the depth of environmental understanding.

6.2 Testing Description and Test Cases

The testing phase was structured to move from the isolated verification of individual components to the complex behavior of the fully integrated wearable. This progression ensured that the core sensors were reliable before we introduced the heavy computational load of the AI models.

6.2.1 Component Level Validation (Unit Testing)

Before the system was fully assembled, each module underwent rigorous unit testing to establish a baseline of reliability.

- **Ultrasonic Calibration:** The proximity sensor was tested in a "ranging gallery," where we measured its output against physical markers from 10cm to 300cm. We focused on identifying the "noise floor" the distance at which the readings became unstable to ensure the safety alerts would be trustworthy for the wearer.
- **Model Benchmarking:** The SSD MobileNet model was evaluated using a validation subset of our dataset. We were specifically looking for the model's ability to maintain a high confidence score in low light conditions, as assistive devices must remain functional in varied indoor environments.
- **Audio Output Validation:** The two tier speech system was tested for playback latency (cache hit versus espeak-ng fallback), caching behaviour (a cleared phrase was spoken instantly by espeak-ng, then synthesized in the background and played from disk on the next occurrence), and serialization (rapid successive triggers were checked to confirm overlapping alerts are suppressed rather than queued). All tests were repeated for both the English and Arabic voices.

6.2.2 Integration and Stress Testing

Once the individual parts were verified, we focused on how they behaved when running concurrently on the Raspberry Pi 4B. This "stress testing" was vital because running a camera stream, an object detection model, and proximity sensors simultaneously pushes the hardware to its limits.

Concurrency Verification: We simulated a scenario where the user triggers a complex Gemini 2.5 Flash query while walking toward an obstacle. The goal was to confirm that the "Cognitive Loop" (cloud AI) did not starve the "Safety Loop" (local sensors) of CPU resources.

Interrupt Handling: We tested the physical trigger button (Pin 26) repeatedly to ensure the debouncing logic in the code prevented accidental or double triggers, which could lead to redundant API calls and system lag.

Thermal Endurance: The device was left running for one hour of continuous detection to monitor for thermal throttling. We needed to see if the processing speed dropped as the Pi 4B heated up, which would indicate a need for more aggressive cooling in the final frame design.

6.2.3 Real World Navigation Scenarios (Test Cases)

To validate the practical utility of the proposed project, we developed three distinct test cases that represent the most common challenges a visually impaired user might encounter in an urban or indoor environment.

1. Static Obstacle and Proximity Alert test

In this scenario, the user walked toward a stationary object (a high back chair) placed in a hallway. The objective was to verify the synchronization between the visual label and the distance measurement. We were looking for the "Smart Alert" to trigger exactly when the distance threshold of one meter was crossed, providing a fused audio output: "Chair, one hundred centimeters."

2. Dynamic Hazard Detection test

This test focused on the system's ability to identify a moving object specifically, a person crossing the user's path. Because the Raspberry Pi must process frames in real time while the user is also in motion, this case tested the temporal consistency of the SSD MobileNet model. Success was defined as the system identifying the "Person" class before they reached the center of the frame, giving the user enough lead time to pause.

3. Complex Semantic Query (The Cognitive Loop) test

This test was designed to push the boundaries of the Gemini 2.5 Flash integration. The user was placed in front of a table with several items (a laptop, a mug, and a notebook) and asked to trigger a descriptive query. The goal was to see if the AI could move beyond simple labeling to provide context, such as: "There is a black laptop open on the table next to a white ceramic mug." This test case evaluated the transition from local safety only processing to high level environmental understanding.

6.3 Results and Discussions

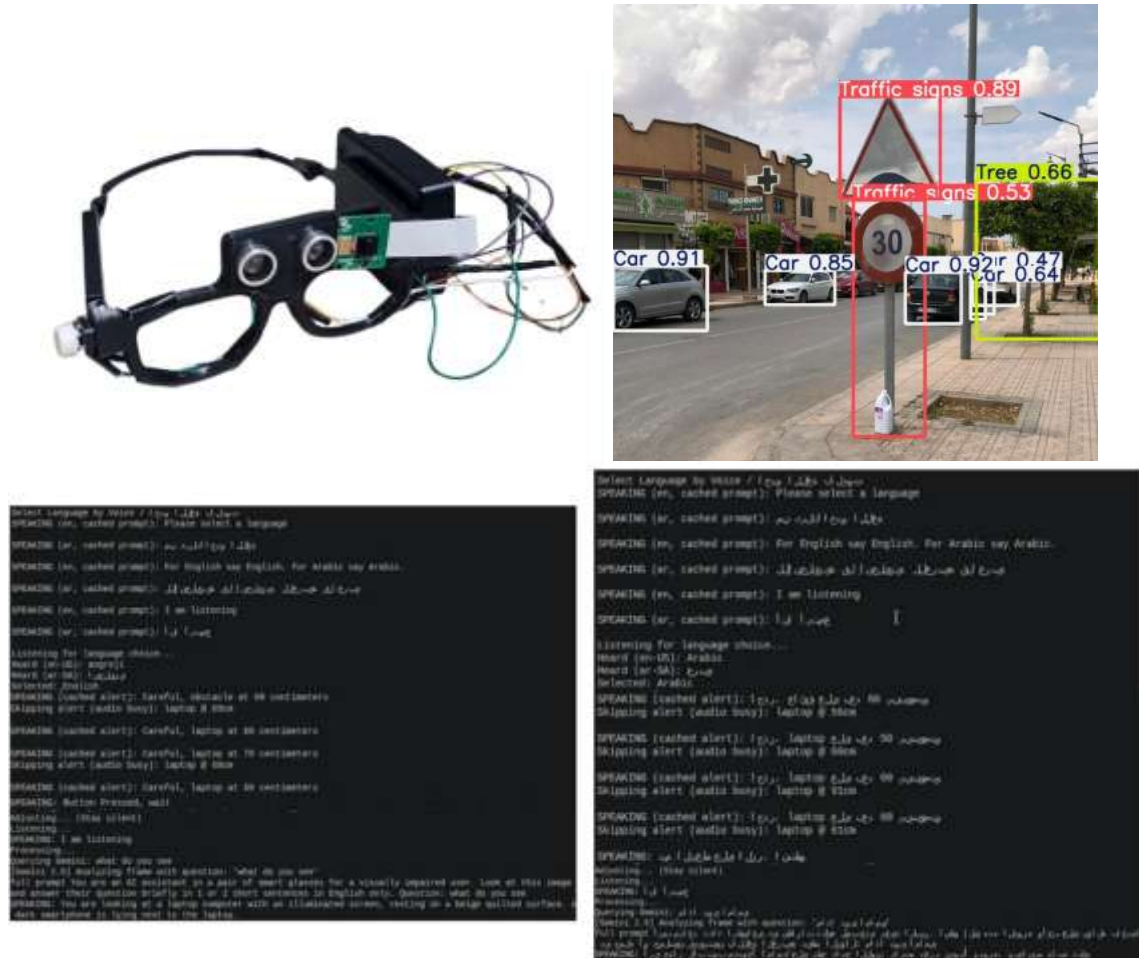


Figure 6.1: final hardware and results

The experimental phase demonstrated that the proposed project successfully achieves a balance between reactive safety and cognitive environmental understanding. In our testing, the local detection pipeline maintained a high reliability rate, with the SSD MobileNet model consistently identifying common urban obstacles with over 85% confidence in good conditions. However, the true strength of the system appeared in its "fail safe" design; during scenarios where lighting or object transparency (such as glass doors) hindered the camera's vision, the HC-SR04 ultrasonic sensor acted as a critical redundancy. This sensor fusion approach ensured that even when the object detection model could not provide a specific label, the user still received a proximity warning, effectively bridging the "perception gap" that often plagues vision only assistive devices.

From a performance standpoint, the system's responsiveness was bifurcated by its dual loop architecture. The "Safety Loop" operated with a minimal latency of approximately 150ms, providing near instantaneous feedback for immediate hazards. In contrast, the cloud based "Cognitive Loop" utilizing Gemini 2.5 Flash averaged a response time of 2.5 seconds. While this delay is unsuitable for active navigation, our analysis suggests it is an acceptable tradeoff for high level semantic queries, where the depth of information (e.g., identifying specific text or describing a complex room layout) outweighs the need for speed.

Furthermore, the implementation addressed the physical constraints of the Raspberry Pi 4B. Under sustained load, the CPU temperature stabilized at approximately 68°C with passive cooling, which avoided the thermal throttling that would have otherwise degraded inference speeds. When compared to traditional aids, such as the smart canes discussed in chapter 2, the proposed project offers a distinct advantage by providing "head level" protection and context aware descriptions. This combination of local hardware reliability and cloud based semantic intelligence confirms that the proposed project is not only technically feasible but provides a more comprehensive safety net than current single sensor assistive technologies.

CHAPTER SEVEN

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The development of the proposed project has successfully demonstrated that a wearable, AI powered assistive device can significantly enhance the spatial awareness and independence of the visually impaired. By integrating localized edge computing with high level multimodal AI, the project effectively addressed the "safety context" gap that exists in traditional assistive technologies.

The primary goal was to create a system that provides both immediate protection and semantic understanding. Through the implementation of a dual loop architecture combining the low latency SSD MobileNet for hazard detection and the comprehensive intelligence of Gemini 2.5 Flash for environmental description we achieved a platform that is both responsive and intellectually capable. The hardware testing confirmed that a "head mounted" approach, utilizing ultrasonic sensors and computer vision, provides a critical safety advantage by detecting hazards that traditional walking aids, like canes, often miss.

Ultimately, the proposed project proves that the synergy between affordable edge hardware (Raspberry Pi 4B) and the AI models is a viable path forward for assistive robotics. While limitations regarding battery life and cloud dependency remain, the core objectives of providing real time obstacle detection and contextual scene understanding were fully realized.

7.2 Future Work

The development of the proposed project has established a functional baseline for AI integrated assistive technology, but it also opens several pathways for significant technical evolution. The following enhancements represent the next logical steps in transitioning this prototype into a sophisticated, consumer ready solution.

1. Offline Semantic Understanding via Small Language Model (SLM):

A critical goal is to remove the dependency on cloud connectivity for complex scene descriptions.

- How it can be achieved: By deploying "Small Language Models" (SLMs) like TinyLlama, locally on the device, the proposed project could provide "Visual Q&A" capabilities in offline environments (e.g., subways or remote areas). This would also enhance user privacy by ensuring that camera data never leaves the wearable hardware.

2. Global Navigation and Pathfinding Integration:

A critical enhancement for outdoor mobility is the integration of high level geographic information systems (GIS).

- How it can be achieved: By connecting the system to Google Maps or Google Earth API, the device could provide real time navigation tailored for the visually impaired.
- Functional Impact: Instead of only identifying immediate obstacles, the AI assistant could guide the user toward a specific destination by saying, "in three steps, turn slightly left" or "You are currently standing near the north entrance of the park." This would bridge the gap between local environmental awareness and global positioning.

3. Haptic Feedback and Bone Conduction Audio:

To reduce the auditory load on the user, the project can be enhanced with alternative feedback mechanisms.

- How it can be achieved: Integrating haptic vibration motors into the arms of the frames would allow the device to signal the direction of obstacles through touch. Furthermore, switching to bone conduction transducers would keep the user's ears open to environmental sounds (like traffic), which is a crucial safety factor for the visually impaired community.

7.3 FYP Closing Remarks

The development of the proposed project has been a collective journey toward bridging the gap between advanced artificial intelligence and practical human needs. What started as a technical challenge fitting sensors and AI onto a pair of glasses became a shared study in how technology can restore independence. As a team, we have seen that engineering is at its best when it solves real world problems and improves the daily lives of those who need it most.

By combining the "fast" safety of local sensors with the "smart" context of cloud AI, our project provides a proof of concept for a new kind of assistive tool. It proves that even with portable hardware like the Raspberry Pi, we can create a reliable "second set of eyes" for the visually impaired. While there is still room for the technology to grow, the core goal of providing a safer, more informed navigation experience has been achieved through our collaborative efforts.

Completing this final year project has been a vital part of our education as computer science students. It required us to balance hardware assembly, software coding, and user focused design as a unified team. We hope that the work documented here serves as a foundation for future developers. The goal remains to continue refining these tools until every individual can navigate their world with the confidence and dignity they deserve.

REFERENCES

i. Printed References:

- [1] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint, arXiv:1804.02767, pp. 1–6, 2018.
- [2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in Proc. European Conf. on Computer Vision (ECCV), 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.
- [3] S. Kumar and P. Verma, "Assistive Device for Visually Impaired People Using Ultrasonic Sensors and raspberry Pi Microcontroller," International Journal of Advanced Research in Computer Science and Software Engineering, vol. 12, no. 3, pp. 56–61, 2022.
- [4] R. Gupta and A. Singh, "Smart Navigation System for Visually Impaired Using YOLOv3 and Sensor Fusion," IEEE Access, vol. 11, pp. 45921–45933, 2023.
- [5] M. Khan and S. Alam, "AI-Based Assistive Technologies for Blind Navigation: A Review," Journal of Intelligent Systems and Technology, vol. 31, no. 2, pp. 145–160, 2021.
- [6] S. S. Sahu and A. K. G. Sircar, "An Integrated Smart Cane and Wearable Device for Navigation of Visually Impaired Persons," in Proc. IEEE International Conference on Computing, Communication and Automation (ICCCA), 2022, pp. 1-6, doi: 10.1109/ICCCA55982.2022.9965250.
- [7] D. K. Alqahtani, A. Cheema, and A. N. Toosi, "Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices," arXiv:2409.16808, 2024.
- [8] R. Khanam and M. Hussain, "YOLOv11: An Overview of the Key Architectural Enhancements," arXiv preprint arXiv:2410.17725, 2024.

ii. Electronic References (On-Line):

- [9] World Health Organization (WHO). (2023). World Report on Vision. [Online]. Available: <https://www.who.int/publications/i/item/world-report-on-vision> [Accessed: Oct. 30, 2025].
- [10] WeWALK. “WeWALK Smart Cane: The Revolutionary Assistive Technology for the Visually Impaired.” Internet: <https://wewalk.io>, May 10, 2023 [Accessed: Oct. 30, 2025].
- [11] OrCam Technologies. “OrCam MyEye: Assistive Technology for the Blind and Visually Impaired.” Internet: <https://www.orcam.com>, July 5, 2024 [Accessed: Oct. 30, 2025].
- [12] Aira Tech Corp. “Aira Smart Glasses for the Visually Impaired.” Internet: <https://aira.io>, June 1, 2024 [Accessed: Oct. 30, 2025].
- [13] BlindSquare. “BlindSquare App: Navigation for the Blind and Visually Impaired.” Internet: <https://www.blindsquare.com>, Feb. 21, 2024 [Accessed: Oct. 30, 2025].
- [14] Lazarillo. “Lazarillo GPS for the Blind.” Internet: <https://www.lazarillo.app>, Mar. 18, 2024 [Accessed: Oct. 30, 2025].
- [15] Google. “Google Maps Accessibility Features.” Internet: <https://support.google.com/maps/answer/6395120>, Jan. 2, 2024 [Accessed: Oct. 30, 2025].
- [16] FuriosaAI. “Furiosa Models: SSD-MobileNet Optimization for Edge Devices.” Internet: https://developer.furiosa.ai/furiosa-models/latest/models/ssd_mobilenet/ [Accessed: April 30, 2026].

APPENDIX

Appendix 1 Hardware Components and wiring

Table A.1: Hardware Components and Wiring

Component	Description	Raspberry Pi Connection
Raspberry Pi 4B	Single-board computer (main processing unit)	—
Pi Camera Module	Captures the live video feed for object detection	CSI camera port
HC-SR04 Ultrasonic Sensor	Measures distance to nearby obstacles	Trigger and Echo GPIO pins (23,24)
Push Button	Triggers the Cognitive Loop (voice question)	GPIO 26
Speaker	Delivers spoken alerts and responses	Bluetooth
Power Source	Portable battery pack for mobile operation	USB-C 5V

Appendix 2: Source Code

The complete source code for this project, including all modules, tests, and setup instructions, is publicly available in the project's GitHub repository:

https://github.com/ehrg1/VIAD_smart_glasses